HW/SW Co-Design - Project Report

Shay Midginski 206391724
Nitzan Behar 209825066

Benchmark: mdp

Part I - Overview:

Description of the benchmark:

1. This benchmark measures the performance of solving a Markov Decision Process (MDP) through iterative value estimation.
2. MDP is a mathematical model for sequential decision making when outcomes are uncertain. It is a type of stochastic decision process, and is often solved using the methods of stochastic dynamic programming. It is commonly described as a graph.
3. The goal in MDP is to find a "good policy" for the decision maker: a function that describes which action the decision maker will choose when in state s. The objective is to choose a policy that will maximize some cumulative function of rewards.
4. The general mathematical model is defined by a 4-tuple (S, A, P_a, R_a):
   (a) S is a set of states. State s is a node in the graph.
   (b) A is a set of actions. Action a is an edge in the graph $(s \overset{a}{\longrightarrow} s')$, the transition from one state to another is done through a choice of action.
   (c) $P_a(s, s')$ is the probability that a transition will be performed from state s to state s' after choosing action a.
   (d) R_a is the reward received after action a is taken to transition from state s to state s'. The reward is generaly a random variable.

   We can see those properties in the next drawing (taken from Wikipedia) where the state space $S = \{s_0, s_1, s_2\}$ (green circles), the actions space $A = \{a_0, a_1\}$ (orange circles), the probability function $P_a(s, s')$ (weights on the edges) and with two rewards R_a (orange arrows).

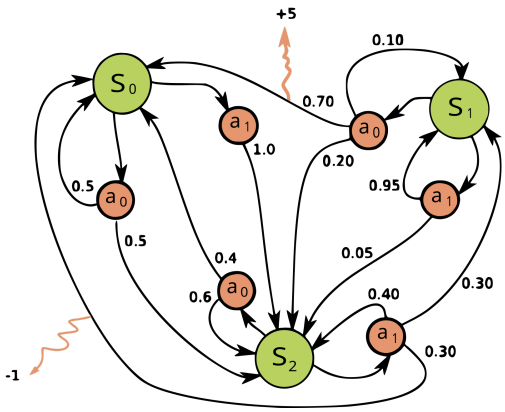


Figure 1: example of a simple MDP

5. In the current benchmark, the MDP is modeled as a simplified turn-based Pokémon battle (player as Charizard vs. opponent as Starmie) and computes the optimal win-probability at the root state via a min/max interval-narrowing iteration over a topologically sorted state graph.



Figure 2: Charizard and Starmie

6. The purpose of this method is to stress numeric computation, exact rational arithmetic, dictionary-heavy graph traversal, and moderately deep recursion/iteration. The workload represents decision-theoretic code and AI rather than the battle simulation itself being the point.

Libraries:

1. The two libraries used in the benchmark are **collections** (namedtuple, defaultdict) and **fractions.Fraction**.
2. **collections.namedtuple** builds every state and data record in the file (*stats_t*, *fixeddata_t*, *halfstate_t*, *attack_stats_t*).
   (a) **namedtuple** is immutable and hashable so battle states built from this type are used as dictionary keys. A mutable object (regular class or dict) could not be hashed or safely reused as a key.
   (b) **namedtuple** supports structural equality (comparison using ==) and *._replace()* for efficient copy-and-modify of states.
3. **collections.defaultdict** is a subclass of Python's built-in dictionary (dict). The difference between them lays in the way they handle missing keys. In standard dictionary, If you try to access or update a key that does not exist, Python will raise a KeyError with a default dict that automatically creates that key and assigns it with a pre-defined default value instead of throwing an error. Used in our code for:
   (a) **defaultdict(Fraction)** for probability distributions - damage and state distributions. This allows code like *dist[key] += p* without first checking membership with coalescing separate paths that land on the same outcome.
   (b) **defaultdict(float) / defaultdict(lambda: 1.0)** - this lets lazy initializations for states that weren't yet processed to a default value without an explicit initialization pass over the whole state space.
4. **fractions.Fraction** provides exact rational arithmetic value for every probability in the code.
   (a) The benchmark's correctness check compares a float result against a hardcoded expected constant and expects a precision level of up to e^{-6} - computing probabilities exactly avoids floating-point rounding drift accumulating across dict merges.
   (b) Conversion to float happens only once, at the very end of the code once the exact value has already been settled.

Data structures:

1. Four **namedtuples** define the domain:
   (a) **stats_t** = namedtuple('stats_t', ['atk', 'df', 'speed', 'spec']) - a combatant's four core battle statuses (attack, defense, speed, special). Reused both for base stats and for stat modifiers. A constant NOMODS = stats_t(0,0,0,0) represents "no modification".
   (b) **fixeddata_t** = namedtuple('fixeddata_t', ['maxhp', 'stats', 'lvl', 'badges', 'basespeed']) - all of a certains combatants data that stays constant for the whole battle: max HP, base stats_t, level, badges held, and base speed.
   (c) **halfstate_t** = namedtuple('halfstate_t', ['fixed', 'hp', 'status', 'statmods', 'stats']) - a combatants current, changing condition: a reference to its fixeddata_t (fixed), current HP, status, active stat modifiers (statmods, a stats_t), and current effective stats after those modifiers are applied. Since namedtuples are immutable, whenever HP changes - a new halfstate_t is produced via ._replace(hp=...) rather than mutating the existing one, which is also what makes every state hashable and usable as a dictionary key.

(d) **attack_stats_t** = namedtuple('attack_stats_t', ['power', 'isspec', 'stab', 'te', 'crit']) - a move's fixed data (base power, whether it's special, STAB eligibility, type-effectiveness multiplier, crit eligibility), stored separately in a dictionary keyed by move name and looked up only when an action is resolved.

2. The battle itself is represented as the standard MDP 4-tuple (S, A, P_a, R_a), where the objective is to find the best policy for Charizard to act on in order to win before he losses.
3. **States (S):** Every state is a tuple called *statep*: (*tag*, *state*) where *state* is a tuple of two **halfstate_t namedtuples** describing both combatants current HP/status and *tag* is an index (int) $\in \{0, 1, 2, 4\}$ that identifies which of four state types it is:
   (a) Type 0 - players decision state. The player (Charizard) has to choose an action. Represented as (0, state),
   (b) Type 1 - opponents chance state and the cost/gain of its move. The players action have been chosen but before the turn plays out: the enemys move is sampled from a fixed probability distribution, and turn order is resolved by comparing speed so the faster attackers damage roll is applied. Represented as (1, state, action).
   (c) Type 2 - second attack. Represents the state after the first attackers move has already been resolved, and the conclusion of the second combatant's action. Represented as (2, state, side, action).
   (d) Type 4 - terminal state. The battle has ended. Only two such states exist: (4, True) for a win and (4, False) for a loss. Both are absorbing - they have no successors and their values are fixed rather than computed.
4. **Actions (A):** Charizard can choose between 'Dig' and 'Super Potion', while Starmie has a fixed probability distribution between 'Water Gun' and 'Bubblebeam'.
5. **Transition probabilities ($P_a(s, s')$):** A dictionary mapping each possible successor statep to its probability. Given that Charizard can choose action, the transition probabilities are for Starmie's actions and is hardcoded in the benchmark's code: $P_{Water_Gun} = \frac{64}{130} = 0.492,\ P_{Bubblebeam} = \frac{66}{130} = 0.508.$
6. **Reward (R):** This model has no per-transition reward - the terminal payoff is fixed directly: winning gains 1 and a loss is costs 0.
7. **Supporting structures:**
   (a) Two dictionaries, *self.min* and *self.max*, hold the lower and upper value bound for every state.
   (b) A set, *self.frozen*, tracks which states have already converged and can be skipped in later passes.
   (c) A list, *stateps*, holds all reachable states in topological order, computed once and reused every pass.

Algorithm:

1. The benchmark solves a stochastic shortest path MDP probelm with no per-step reward, only a terminal payoff (R(win) = 1, R(loss) = 0), using bounded value iteration method.
2. Each full turn of the battle is:
   (a) Type-0 (decision): Charizard picks Dig or Super Potion → edge to type-1, tagged with the chosen action.
   (b) Type-1 (chance - enemy move + first-attacker's damage roll): samples enemy action (Water Gun/Bubblebeam) from fixed probabilities, determines move order by speed, resolves the first attacker's damage distribution → edge either to type-4 (terminal, if first attack was lethal) or to type-2 (carrying the pending second action).
   (c) Type-2 (second attackers damage roll): resolves the second, pending actions damage distribution → edge either to type-4 (terminal, if lethal) or back to a fresh type-0 decision node for the next turn.
   (d) Type-4 (terminal, absorbing): win/loss, values fixed at 1.0/0.0.
3. As stated, the goal is to tell which action the decision maker should choose that will lead him to winning before lossing.
4. The two primary algorithms to solve MDP problems are Policy Iteration and Value Iteration:
   (a) Policy Iteration starts with an arbitrary policy (a function that describes how to choose actions depending on the current state) and iteratively "makes it better" through a two-step process. It tests

the current policy if it satisfies the requirements (win before loss), updates the policy if not, and repeats this loop until the policy cannot be improved anymore.

(b) Value Iteration focuses entirely on finding the best possible scores (values) for each spot on the graph by updating them over and over. Once it figures out the perfect scores, it simply creates a final plan by picking the actions that lead to those high scores.

5. Those methods are iterative, and therefore usually run on a fixed number of iterations or in a stochastic way (until the change is smaller than some threshold). In the current benchmark, we run bounded variation of Value Iteration, the algorithm tracks a lower and upper bound, *self.min[s]* and *self.max[s]*, updated by the Bellman equations independently every sweep until the min and max value in a specific state converge to a value that is in the tolerane range (*dmax - dmin* $\leq$ *tolerance*).
6. The Bellman equations:

   (a) At a decision state (Type-0): $V^*(s) = \max_{a \in A} \sum_{s' \in S} P_a(s, s') \cdot V^*(s')$. The player picks the action maximizing expected win probability.

   (b) At a chance state (Type-1): $V^*(s) = \sum_{s' \in S} P_a(s, s') \cdot V^*(s')$. This is a probability-weighted average, since there is no choice, only a sampled outcome from a fixed distribution.

   (c) Boundary conditions: $V^*(win) = 1,\ V^*(loss) = 0$.
7. In the benchmark's code, *evaluate()* only ever applies the Bellman equations but the optimal action at each decision state is never extracted or stored. It is only implicit in whichever successor achieves the max.
8. States are processed in the order of a topologically sorted list (DFS-based) so that a state's successors are updated before the state itself within the same sweep.
9. Once a state's bounds converge (*dmin[sp]* >= *dmax[sp]*), it is collapsed to their average and added to self.frozen, so future sweeps skip it entirely.
10. The loop exits once *dmax[initial_statep]* - *dmin[initial_statep]* $\leq$ *tolerance*, and the returned value is their average.
11. This is a dynamic-programming algorithm: it exploits optimal substructure (a state's value is a function of its successors' values via the Bellman equation) and overlapping subproblems (many states share successors, so their values are reused rather than recomputed).

Part II - Initial Analysis:

Basic benchmark test run:

```
pyperformance run -b mdp -o mdp_basic_results.json
```

1. This command executed 20 independent measurement processes of the MDP benchmark (and one calibration run).
2. The results were consistent, with a mean runtime of 4.2497 seconds, a median of 4.2477 seconds, and a standard deviation of only 0.01196 seconds (approximately 0.3% relative variation).
3. This low variance indicates a stable testing environment.
4. This measurment will act as our base in the optimization process.

Performance analysis:

1. We have ran perf record on the entire pyperformance run using:

   ```
   sudo perf record -F 99 -g --call-graph dwarf -e cpu-clock -- python3.12-dbg -m
   pyperformance run --bench mdp
   sudo perf report --stdio --no-inline > mdp_perf_report_312.txt
   ```

2. Let's sort the results by %self using:

   ```
   grep "^    [0-9]" mdp_perf_report_312.txt | sort -k2 -rn
   ```

3. The results show us 4 dominant function:

   ```
   97.80%    16.24%    python    python3.12d    [.] _PyEval_EvalFrameDefault
   36.18%    13.86%    python    python3.12d    [.] tuplehash.lto_priv.0
   37.00%    11.78%    python    python3.12d    [.] PyObject_Hash
   11.20%    11.20%    python    python3.12d    [.] long_hash.lto_priv.0
   ```

   (a) The first column is %children and the second column is %self.

   (b) The 5th function in this list has %self $< 2\%$.

4. **_PyEval_EvalFrameDefault** is CPython's core bytecode interpreter. Every Python operation runs through this function. This is why its %Children = 97.8%. This interpreter loop has a fixed overhead for a cost in almost every Python run, so we don't see any reason to dig into that optimization path.
5. On the other hand, the other 3 functions: **tuplehash.lto_priv.0**, **PyObject_Hash** and **long_hash.lto_priv.0** are CPython implementation for hashing Python object (such as tuples and long as we can see here) and consume about 36.84% of the running time. This points to the benchmark repeatedly hashing tuples of integers as dict keys, which is consistent with our knowledge about MDP implementation with (state, action) tuples.
6. If we look further on the call tree of **_PyEval_EvalFrameDefault**, **tuplehash.lto_priv.0** or **PyObject_Hash**, we see a deep recursive calls like this:



Figure 3: Screenshot from mdp_perf_report_312.txt

long_hash.lto_priv.0 doesn't invoke further calls.

7. We see that **tuplehash.lto_priv.0** calls **PyObject_Hash** and vice versa, which is perf's way of showing the same two functions calling each other repeatedly for a tuple with multiple hashable fields. This is most likely the multi-field (state, action) or similar composite key from the mdp benchmark code. The depth of recursion suggests the key tuple has several elements, each contributing its own **long_hash.lto_priv.0** call.
8. This suggests that the bottleneck isn't one expensive hash call, but a chain of nested ones per dict lookup. Our current strategy should be trying to reduce the amount of hashing operations or the recursion depth.
9. In order to pinpoint the bottleneck area in the benchmark code itself, we want to see which Python function invokes this recursion.
10. The limitation of this run are worth mentioning: this perf record captures pyperformances entire benchmark invocation, which forks 21 independent Python processes. perf's call-tree percentages are summed across all 21 processes, including per-process startup/teardown that isn't part of the actual MDP workload, with no per-run isolation. Currently, we cannot tell which Python function in the MDP benchmarks own source is driving the hashing bottleneck.
11. Therefore, we will run another perf record against a single invocation of bm_mdp instead of the full pyperformance run wrapper. We will enable actual Python function resolution using CPython 3.12 added built-in perf support, PYTHONPERFSUPPORT=1, that injects per-call trampoline frames so perf can resolve and display real Python function names:

```
export PYTHONPERFSUPPORT=1
sudo -E perf record -F 99 -g --call-graph dwarf -e cpu-clock -o perf_single.data -
- python3.12-dbg /usr/local/lib/python3.12/dist-packages/pyperformance/data-
files/benchmarks/bm_mdp/run_benchmark.py --worker -l1 -w1 -n3
sudo perf report -g -i perf_single.data --no-inline >
mdp_perf_report_312_single.txt
```

12. As a sanity check, let's sort the results by %self using:

```
grep "^    [0-9]" mdp_perf_report_312_single.txt | sort -k2 -rn
```

13. The results show us the same picture as the multi-process run, which is a good signal:

```
97.59%    16.18%  python3.12-dbg  python3.12d    [.] _PyEval_EvalFrameDefault
```

```
34.79%    13.14%  python3.12-dbg  python3.12d    [.] tuplehash.lto_priv.0
35.51%    11.79%  python3.12-dbg  python3.12d    [.] PyObject_Hash
10.51%    10.51%  python3.12-dbg  python3.12d    [.] long_hash.lto_priv.0
```

14. In addition, we will sort the Python symbols by %children, to see which Python function invokes most of the work, using:

```
grep "^    [0-9]" mdp_perf_report_312_single.txt | grep "py::" | sort -k1 -rn
```

15. The results below show us which function performs most of the work:

```
26.62%     0.00%     python3.12-dbg     [JIT] tid 2490     [.]
py::Battle.evaluate:/usr/local/lib/python3.12/dist-packages/pyperformance/data-
files/benchmarks/bm_mdp/run_benchmark.py

17.62%     0.12%     python3.12-dbg     [JIT] tid 2490     [.] py::Battle.evaluate.
<locals>.<genexpr>:/usr/local/lib/python3.12/dist-packages/pyperformance/data-
files/benchmarks/bm_mdp/run_benchmark.py

15.28%     0.19%     python3.12-dbg     [JIT] tid 2490     [.] py::Battle.evaluate.
<locals>.<genexpr>:/usr/local/lib/python3.12/dist-packages/pyperformance/data-
files/benchmarks/bm_mdp/run_benchmark.py
```

16. The two **Battle.evaluate.<locals>.<genexpr>** entries (17.62% and 15.28% of children) are the generator expressions inside *evaluate()* per-sweep update: *max(dmin[sp2] for sp2 in self.getSuccessors(sp)) / max(dmax[sp2] ...)* at decision states, and *sum(dmin[sp2]*p for sp2, p in self.getSuccessors(sp)) / sum(dmax[sp2]*p ...)* at chance states.
17. Together they account for the bulk executed by *evaluate()* - 26.62%. Each generator re-iterates a state's successor list and performs a fresh *dmin[sp2]*/*dmax[sp2]* dict lookup per successor, hashing the full statep tuple every time.
18. This is exactly why **tuplehash**/**PyObject_Hash**/**long_hash** collectively consume over a third of the total runtime.
19. This narrows the optimization target: the cost isn't evaluate's control flow itself, but the repeated tuple-hashing incurred by re-looking-up *self.min*/*self.max* for every successor, on every sweep.

Flame graphs:

We have genereated a flame graph for the single process run of the benchmark using:

```
sudo perf script -i perf_single.data --no-inline > mdp_single.perf
~/FlameGraph/stackcollapse-perf.pl mdp_single.perf > mdp_single.folded
~/FlameGraph/flamegraph.pl mdp_single.folded > mdp_312_single_flamegraph.svg
```

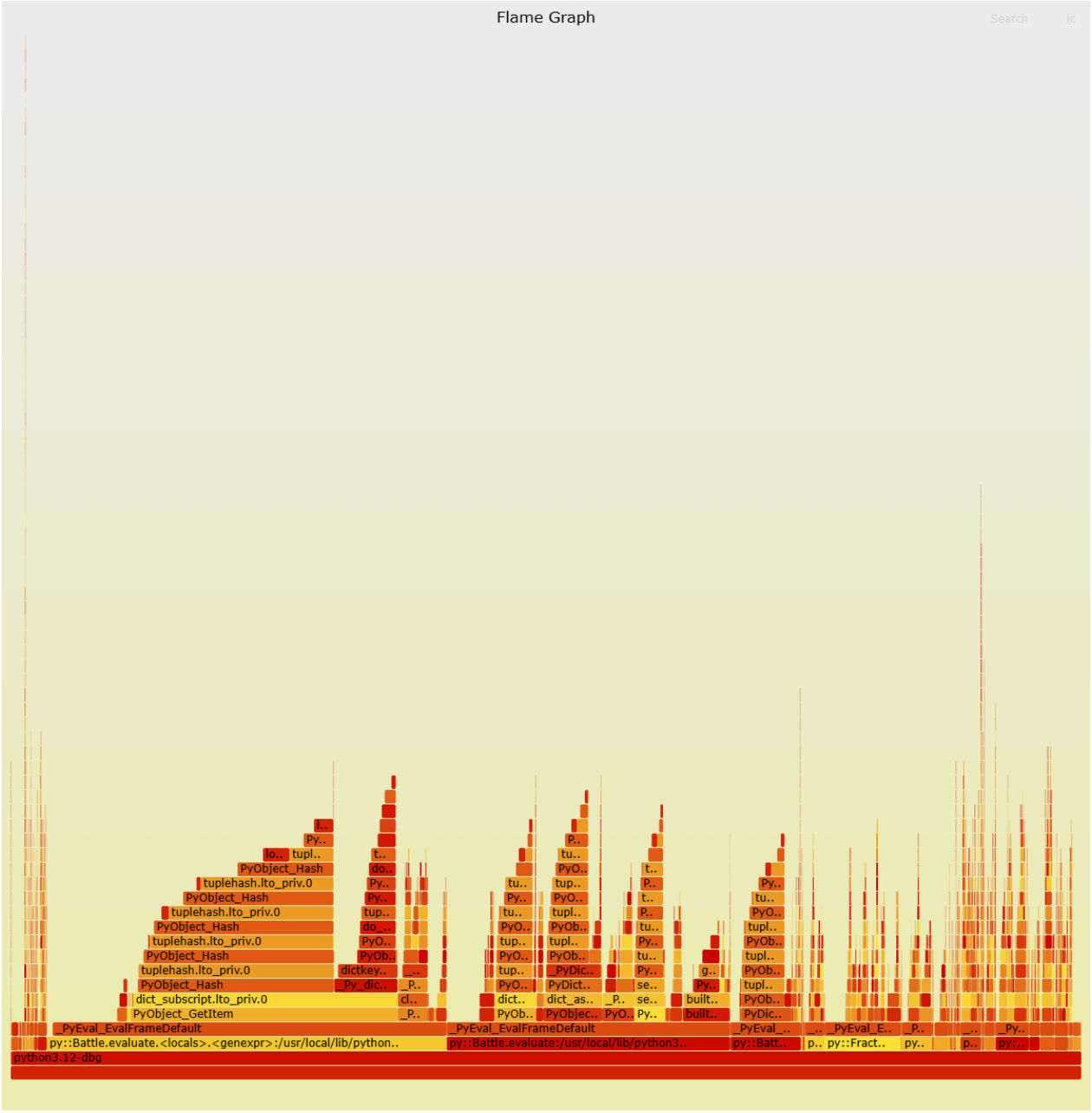


Figure 4: mdp_312_single_flamegraph.svg

1. The three dominant components in this flame graph match the functions already flagged as the bottleneck in the perf output.
   (a) **Battle.evaluate.<locals>.<genexpr>** is the widest component in the graph. This is the generator expression inside *evaluate()* max and sum calls. This confirms that the two <**genexpr**> entries reported at 17.62% and 15.28% of children correspond to real, substantial width in the flame graph.
   (b) **Battle.evaluate** itself (the function body, not the generator expressions) is the second-widest component.
   The flame graph shows that time is concentrated exactly in the two places the sweeps reads (the genexpr) and writes (the assignment lines).

   (c) **Battle.getSuccessors** is the third-largest hashing component, corresponding to the *self.successors[statep]* memoization cache read/write. This gives a concrete location for part of the **tuplehash**/**PyObject_Hash** cost: every call to *getSuccessors()* hashes a *statep* tuple to check the cache, in addition to the hashing already happening inside evaluate's own *dmin*/*dmax* lookups. This means the recursive hash observed in the perf is produced by at least two distinct call sites, not one.
2. Together, these components account for most of the graphs total width, which is consistent with the conclusion that the bottleneck is not one expensive hash call, but a chain of nested hashing operations spread across the per-sweep value lookups, all keyed by the same multi-field *statep* tuples.
3. A secondary, non-hashing cost marks Fractions as the next to come bottleneck, after the hashing functions.
4. All remaining components are minor.

Profiling data:

1. the bottleneck is **Battle.evaluate()** - everything else runs once per battle setup, so it's not worth optimizing. The areas we think will improve the performance are:
   (a) **Tuple-keyed dict lookups:** Every iteration of the convergence loop does *dmin[sp]*, *dmax[sp]* where *sp* is a deeply nested tuple ((int, (halfstate_t, halfstate_t, int), ...)). Hashing a nested tuple recomputes a hash by walking into namedtuples each time, which is much slower than an int index into a list. This runs O(#statep × #iterations) times.
   (b) **Redundant *self.getSuccessors(sp)* calls:** Called twice per non-choice node per outer iteration (once for the dmin line, once for dmax), even though the successor list for a given *sp* never changes after the first computation. It's memoized in *self.successors*, so the second call is just a dict lookup that wastes hashing operations.
   (c) **Generator re-creation and re-iteration:** *max(dmin[sp2] for sp2 in self.getSuccessors(sp))* and the matching *dmax* line iterate the same successor list twice, building two separate generator objects, instead of walking the successors once and updating both accumulators together.
   (d) ***sp in frozen* check on tuples:** Same tuple-hashing cost as (a), done once per state per outer iteration just to check if the state can be skipped.

Part III - Optimizations:

Description of improvements made:

1. We found that the bottleneck is exclusively in *evaluate()* and that everything else in *Battle* runs once per *Battle* setup and is out of scope.
2. We shall improve the four points mentioned in part II. All four suggestions share one root fix: *dmin*, *dmax*, and *frozen* were keyed by *statep*, a deeply nested **namedtuple**. Hashing that key means recursively walking into every nested *halfstate_t* → *fixeddata_t* → *stats_t* and hashing each field. The fix resolves each *statep* to a flat integer index once, right after *topoSort()* returns, and replaces every subsequent dict/set operation in the convergence loop with an O(1) list operation on that index.

3. **Tuple-keyed dict lookups:**

   Before:

```
dmin, dmax, frozen = self.min, self.max, self.frozen
stateps = topoSort([initial_statep], self.getSuccessorsList)
...
while dmax[initial_statep] - dmin[initial_statep] > tolerance:
   ...
   for sp in stateps:
            ...
       dmin[sp] = max(dmin[sp2] for sp2 in self.getSuccessors(sp))
       dmax[sp] = max(dmax[sp2] for sp2 in self.getSuccessors(sp))
```

   After:

```
dmin, dmax, frozen = self.min, self.max, self.frozen
stateps = topoSort([initial_statep], self.getSuccessorsList)

# Resolve every state to a flat array index once, so the convergence
# loop below never has to hash a deeply nested statep tuple again
index = {sp: i for i, sp in enumerate(stateps)}
n = len(stateps)

dmin_a = [dmin[sp] for sp in stateps]
dmax_a = [dmax[sp] for sp in stateps]
frozen_a = [sp in frozen for sp in stateps]

initial_i = index[initial_statep]
...
while dmax_a[initial_i] - dmin_a[initial_i] > tolerance:
    for i in range(n):
        ...
        dmin_a[i] = ...   # plain list write, no hashing
        dmax_a[i] = ...
```

   Why this is faster:

   stateps, the complete set of reachable states, is computed once by *topoSort()* before the loop starts, and nothing after that point adds or removes states. The original code re-hashes every *statep* on every single outer iteration just to find where in *dmin*/*dmax* its value lives, paying the nested-tuple hash cost repeatedly for data whose set of keys never changes. Building *index* once pays that hash cost exactly O(#statep) times.

Every read/write for the rest of the function's life is a direct memory offset, no hash function.

Cost removed:

$O(\#statep \times \#iterations)$ of nested-tuple heavy hashing operations $\rightarrow$ $O(\#statep)$ one-time hashes + $O(\#statep \times \#iterations)$ of $O(1)$ list accesses.

4. **Redundant *self.getSuccessors(sp)* calls:**

Before:

```
if sp[0] == 0:
    # choice node
    dmin[sp] = max(dmin[sp2] for sp2 in self.getSuccessors(sp))   # call #1
    dmax[sp] = max(dmax[sp2] for sp2 in self.getSuccessors(sp))   # call #2
else:
    dmin[sp] = sum(dmin[sp2] * p for sp2, p in self.getSuccessors(sp))  # call #1
    dmax[sp] = sum(dmax[sp2] * p for sp2, p in self.getSuccessors(sp))  # call #2
```

After:

```
# Successor edges are already memoized in self.successors (topoSort
# visited every state via getSuccessorsList), so this just resolves
# them to indices once instead of on every iteration.
succ = [None] * n
for i, sp in enumerate(stateps):
    if frozen_a[i]:
        continue
    raw = self.getSuccessors(sp)
    if sp[0] == 0:
        succ[i] = (True, [index[s] for s in raw])
    else:
        succ[i] = (False, [(index[s], p) for s, p in raw])

# ... inside the while loop, per state i:
is_choice, edges = succ[i]     # plain list read — no getSuccessors call at all
```

Why is this faster:

getSuccessors(sp) is already memoized in *self.successors* so for a given *sp* its result never changes after the first computation but each outer iteration calls it twice per state (once building the *dmin* expression and once building *dmax*), on both the choice node and non-choice node branches, and both calls hash *sp* again to do *self.successors[statep]*. Even a cache hit pays the full nested-tuple hash. The fix resolves every state's successor edges to plain index tuples exactly once, before the while loop starts. Inside the loop there is no call to *getSuccessors* at all.

Cost removed:

$2 \times O(\#statep \times \#iterations)$ *self.successors* dict lookups $\rightarrow$ $O(\#statep)$ one-time lookups.

5. **Generator re-creation and re-iteration:**

Before:

```
min[sp] = max(dmin[sp2] for sp2 in self.getSuccessors(sp))   # walks successors once
dmax[sp] = max(dmax[sp2] for sp2 in self.getSuccessors(sp))  # walks the SAME
successors
```

```
dmin[sp] = sum(dmin[sp2] * p for sp2, p in self.getSuccessors(sp)) # walks edges
once
dmax[sp] = sum(dmax[sp2] * p for sp2, p in self.getSuccessors(sp)) # walks the SAME
edges
```

After:

```
is_choice, edges = succ[i]
if is_choice:
    mn = mx = None
    for j in edges:
        dj_min = dmin_a[j]
        dj_max = dmax_a[j]
        if mn is None or dj_min > mn:
            mn = dj_min
        if mx is None or dj_max > mx:
            mx = dj_max
    dmin_a[i] = mn
    dmax_a[i] = mx
else:
    mn = mx = 0.0
    for j, p in edges:
        mn += dmin_a[j] * p
        mx += dmax_a[j] * p
    dmin_a[i] = mn
    dmax_a[i] = mx
```

Why is this faster:

The original code builds two separate generator objects over the identical successor/edge list - one folded into max/sum for *dmin*, and the other for *dmax*, so every list gets walked twice in every state and every iteration, with two generator frames allocated and torn down where one loop would do. This is the only bottleneck that flat-indexing alone doesn't fix (it's an iteration-count problem, not a hashing problem).

Cost removed:

2× iteration + 2× generator construction/teardown per state per iteration → 1× iteration, 0 generator objects.

6. ***sp in frozen* check on tuples:**

Before:

```
for sp in stateps:
    if sp in frozen:      # hashes the full nested statep to probe the set
        continue
    ...
    if dmin[sp] >= dmax[sp]:
        dmax[sp] = dmin[sp] = (dmin[sp] + dmax[sp]) / 2
        frozen.add(sp)     # hashes statep again to insert
```

After:

```
frozen_a = [sp in frozen for sp in stateps]   # built once, before the loop

for i in range(n):
```

```
    if frozen_a[i]:        # plain list index — no hashing
        continue
    ...
    if dmin_a[i] >= dmax_a[i]:
        dmax_a[i] = dmin_a[i] = (dmin_a[i] + dmax_a[i]) / 2
        frozen_a[i] = True   # plain list write — no hashing
```

Why is this faster:
frozen is a set keyed by the same nested *statep* tuples, so both *sp in frozen* and *frozen.add(sp)* pay the recursive-hash cost just to answer a binary yes/no question. A boolean list indexed by the same integer identifier answers it with a single memory read, and marking a state converged is a single memory write without using hash.

Cost removed:
O(#statep × #iterations) of hash operations → O(#statep × #iterations) O(1) of list reads/writes.

Part IV - Performance Comperison:

1. We ran perf record on the optimized mdp code using:

```
pyperformance run --manifest MANIFEST -b mdpOptimized -o mdp_optimized_results.json
```

   The results were consistent, with a mean runtime of 1.180661 seconds, a median of 1.179291 seconds, and a standard deviation of only 0.007334 seconds.

2. Now we will compare the optimized results to the basic run:

```
pyperformance compare mdp/mdp_basic_results.json mdp/mdp_optimized_results.json
```

   The results:

```
### mdp ###
Mean +- std dev: 4.25 sec +- 0.01 sec -> 1.18 sec +- 0.01 sec: 3.60x faster
Significant (t=1694.60)
```

3. This is clear improvement that achived 72.2% reduction in runtime. That confirms that our bottleneck analysis was correct.
4. We do see that is the memory usage is about 1.9 MB higher in the optimized version (29.4-29.9 MB vs 31.3-31.6 MB) which is consistent with the fix trading dict/set overhead for the extra index dict and extra four flat Python lists (dmin_a, dmax_a, frozen_a, succ). This is a small and expected memory/speed tradeoff.
5. We ran another perf record run on the optimized data, using:

```
sudo perf record -F 99 -g --call-graph dwarf -e cpu-clock -- python3.12-dbg -m
pyperformance run --manifest MANIFEST -b mdpOptimized
sudo perf report --stdio --no-inline > mdp_perf_report_312_opt.txt
grep "^    [0-9]" mdp_perf_report_312_opt.txt | sort -k2 -rn
```

   If we sort the results by %self we see:

```
98.43%    28.89%    python    python3.12d    [.] _PyEval_EvalFrameDefault
15.69%     1.42%    python    python3.12d    [.] _Py_Dealloc
34.33%     0.43%    python    python3.12d    [.] binary_op1
```

   That means that **_PyEval_EvalFrameDefault**, CPython's core bytecode interpreter, still dominates runtime, but every other function has %seld $< 2\%$. The hashing functions are no longer the bottleneck.

Part V - Hardware Acceleration Proposal:

The target for hardware acceleration is the convergence loop within Battle.evaluate(). This loop accounts for 48–64% of the execution time even after software optimizations removed redundant dictionary hashing.

Acceleration Justification

The proposed hardware accelerator targets the evaluate convergence loop because it is fundamentally a highly repetitive sparse matrix-vector multiplication and max-reduction kernel that accounts for a huge portion of the benchmark's execution time, even after aggressive software optimizations. In pure Python, evaluating this kernel requires thousands of scalar multiply-add and compare operations, each burdened by severe interpreter bytecode-dispatch and float-boxing overhead.

By offloading this task to a dedicated hardware datapath, we eliminate the software middleman entirely, allowing the circuit to compute one multiply-accumulate or compare operation per clock cycle. For a typical state graph of roughly 5,000 edges, a modest 200 MHz hardware design can complete a full convergence sweep in approximately 125 microseconds. Compared to the tens to hundreds of milliseconds required by the Python implementation, this dedicated hardware approach provides an estimated one to two orders of magnitude speedup, representing the logical next step once software-level optimizations are exhausted.

Data Layout and Hardware/Software Interface

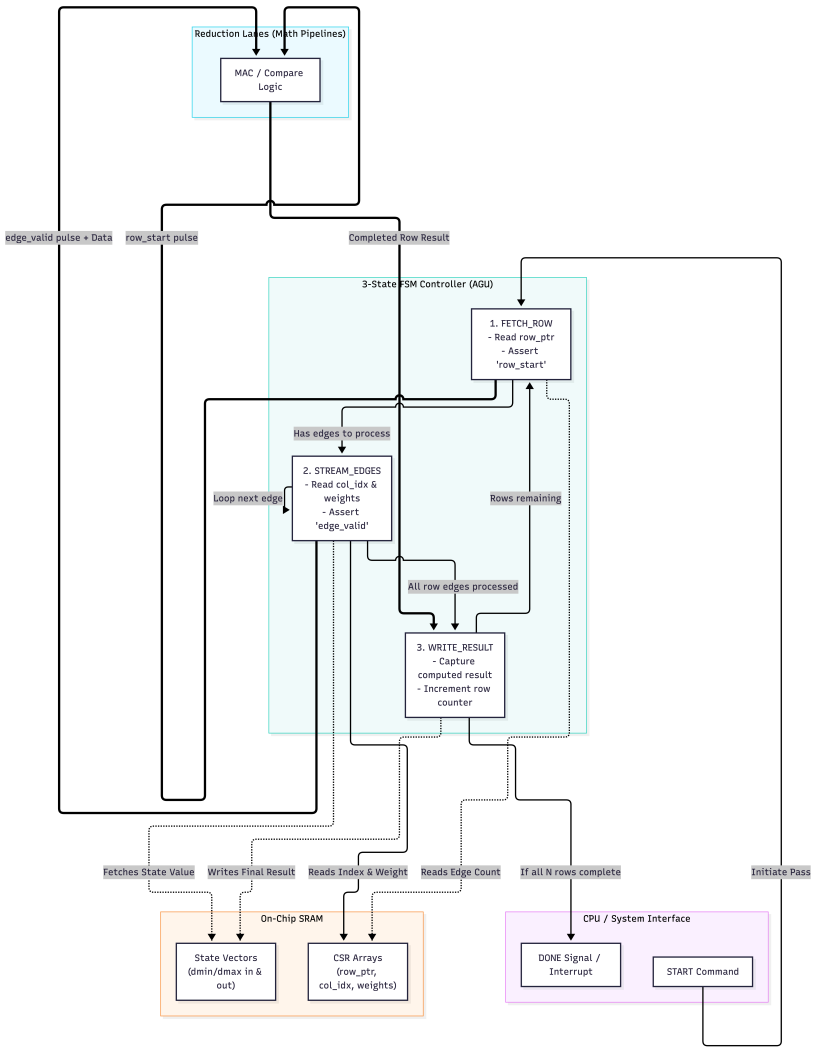
To feed the hardware efficiently, the Python driver must pack the state graph into a Compressed Sparse Row (CSR) layout - a format the accelerator can easily stream:

- row_ptr[n+1]: Cumulative edge count per state.
- col_idx[total_edges]: Flattened successor-state index for every edge.
- weight[total_edges]: Edge probability (forced to 1.0 for choice nodes to utilize the same multiplier).
- mode[n]: 1 bit per state (0 for weighted-sum chance nodes, 1 for max choice nodes).

The CPU and accelerator communicate using memory-mapped registers and direct memory access (DMA). The CPU performs a one-time DMA transfer of the CSR structure into the accelerator's on-chip SRAM.
The working values (dmin and dmax) stay on the accelerator between iterations. For each loop iteration, the CPU simply triggers a "START" register, waits for a "DONE" signal, and swaps the input and output memory pointers for the next pass. The CPU only handles the lightweight scalar math to check if the final values have converged.

Top-Level Integration and Control Logic

To manage the execution and data flow between the memory and the math pipelines, the accelerator employs a 3-state Finite State Machine (FSM). This controller acts as the Address Generation Unit (AGU), ensuring the reduction lanes are fed continuously without sitting idle.



The FSM dictates the internal logic and operation of the accelerator by cycling through these three specific states:

- FETCH_ROW: The FSM begins by reading the row_ptr array to determine the total number of edges associated with the current state. It then asserts a row_start signal to the reduction lanes, which resets their accumulators to zero to prepare for a new computation cycle.
- STREAM_EDGES: The FSM iterates through the active row's connections, indexing from row_ptr[i] to row_ptr[i+1]-1. It uses the col_idx array to fetch the correct input values (dmin_in or dmax_in) and edge weights from SRAM. For each successful fetch, it asserts the edge_valid signal, which feeds the data into the pipelined datapath for calculation.
- WRITE_RESULT: Once all edges for the current row have been streamed and processed, the FSM captures the final value from the reduction lane. It writes this computed result back into the SRAM as the updated dmin_out or dmax_out value, increments the row index, and loops back to FETCH_ROW for the next state.

Once the FSM has processed all N rows in the graph, it asserts the top-level DONE register, which signals the host CPU that the current convergence pass is complete.

Inputs and Outputs

All numerical data within the accelerator uses a 32-bit fixed-point format (Q0.32), which provides more than enough precision to satisfy the benchmark's required error tolerance.

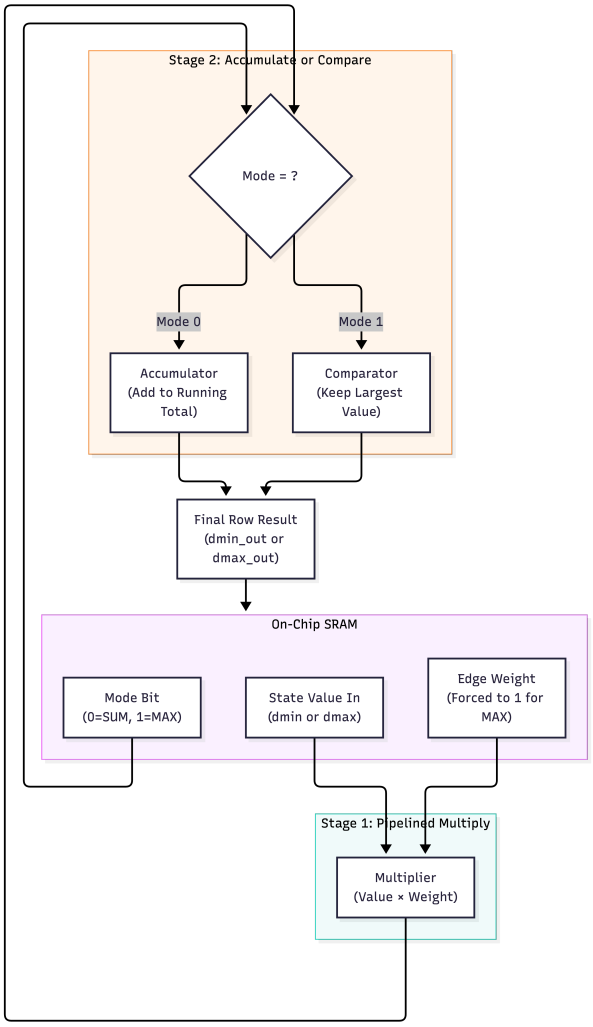
Inputs:

- Graph Structure: The four CSR layout arrays (row_ptr, col_idx, weight, mode) loaded once into the accelerator's memory via DMA.
- Working State: The current state value vectors (dmin_in and dmax_in), which remain resident in the on-chip SRAM.
- Control Signals: Setup registers (memory base addresses and total state count) and a START command triggered by the CPU.

Outputs:

- Updated State: The newly computed dmin_out and dmax_out vectors. These stay in the device memory to act as inputs for the next pass, and are only read back by the CPU once the final values have converged.
- Status Signal: A DONE flag or interrupt sent back to the CPU to notify it that the current iteration is complete.

Hardware Architecture



Verilog Implementation

The implementation consists of a 2-stage pipelined reduction lane that performs either a multiply-accumulate or a compare-select based on the row's mode. The hardware description for the core datapath is attached.

Performance, Area, and Power Trade-offs

- Performance vs. Frequency: The design targets 150-250 MHz. While slower than a CPU clock, it achieves massive throughput by processing one deterministic MAC/compare per cycle with zero interpreter overhead.
- Area vs. Scalability: The datapath is highly compact (two pipelined multipliers and 64-bit accumulators). The primary area cost is the on-chip SRAM to hold the working vectors. This is scalable for the current benchmark's state count (low thousands of entries), but scaling to vastly larger MDP problems would require a dedicated gather cache to maintain pipeline feed rates.
- Power: This fixed-function, mostly-pipelined datapath draws significantly less dynamic power per operation than a general-purpose core fetching and decoding Python bytecode. The main power draw will stem from the SRAM accesses and bus activity.

Part VI - Conclusion:

In conclusion, addressing the computational bottlenecks of the MDP benchmark relies on a two-tiered approach that combines algorithmic restructuring with dedicated hardware offloading. By replacing expensive, nested-tuple dictionary hashing and redundant generator iterations with flat integer indexing and array lookups, the software optimizations successfully eliminated the primary CPython overhead, yielding a 3.60x speedup and a 72.2% runtime reduction.

To bypass Python's remaining built-in performance limits, the proposed hardware accelerator offloads the core evaluate convergence loop into a Compressed Sparse Row (CSR) layout processed by a custom pipelined datapath. Driven by a dedicated FSM operating at roughly 200 MHz, this 32-bit fixed-point architecture computes one deterministic multiply-accumulate or compare operation per clock cycle, delivering an estimated one to two orders of magnitude in additional speedup and transforming a heavily CPU-bound dynamic programming workload into a highly efficient, low-power computational pipeline.